

Rapport d'Audit de Performance

MoteurEchecs — Moteur d'échecs Minimax / Alpha-Beta, sujet libre du projet noté

Module *Optimisations & Performances Backend* — Sup de Vinci, RNCP Bloc 4, Activité 13

Formateur : Christophe Lecroq | Version : brouillon v1 (document de travail, Étapes 1 à 9) | Date : 24 septembre 2026

Langage : Java 21 | Outils : Maven, JUnit 5, JFR (Java Flight Recorder), Hyperfine, JOL, async-profiler

Sommaire

1. Introduction & méthodologie générale
2. **Phase 1 — Analyse factuelle**
 - a. Protocole de mesure
 - b. Synthèse consolidée des 9 étapes (snapshots de versions)
 - c. Flamegraphs réels (captures d'écran)
 - d. Empreinte mémoire — méthodologie et résultats détaillés
 - e. Calcul formel de la loi d'Amdahl
3. **Phase 2 — Explication, compromis & conclusion**
 - a. Choix de solutions, étape par étape
 - b. Influences (cours, barème, référence externe)
 - c. Compromis assumés — liste honnête
 - d. Conclusion : pourquoi cet état est la meilleure solution dans le temps imparti
4. Annexes & reproductibilité

1. Introduction & méthodologie générale

Ce document audite les optimisations appliquées au moteur d'échecs **MoteurEchecs**, sujet libre choisi pour le projet noté, en miroir méthodologique du TP fil rouge **HashBreaker** (craqueur SHA-256) traité en parallèle pendant le même module. Neuf étapes ont été réalisées à ce jour, documentées au fil de l'eau dans `process/` (journal chronologique) et `syntheses/` (résumés autonomes), plutôt que reconstruites a posteriori.

Discipline suivie de bout en bout :

- **Macro avant micro** — réduire la complexité algorithmique avant d'optimiser l'exécution d'un travail donné (cours J1_AM). L'élagage alpha-beta (Étape 2) et le tri des coups (Étape 8) sont macro ; le zéro-allocation et la pré-allocation (Étapes 4, 6) sont micro.
- **Toujours mesurer, jamais supposer** (cours J2_PM) — chaque optimisation est précédée d'une mesure "avant" et suivie d'une mesure "après", avec Hyperfine (temps) et JFR (allocations, CPU, mémoire).

- **Profiler avant d'optimiser** — le tri des coups (Étape 8) n'a pas été choisi au hasard : il découle directement du flamegraph réel de l'Étape 7.
- **Résultats rapportés tels quels**, y compris modestes ou contradictoires (Étape 6 : gain $\times 1,11$ seulement ; Étape 8 : écart entre le gain en positions et le gain en temps ; dérive de baseline entre deux sessions de mesure).

PHASE 1 — ANALYSE FACTUELLE

2.a Protocole de mesure

Temps d'exécution — Hyperfine

Toutes les mesures de temps utilisent `hyperfine --warmup 1 --runs 5` sur un point d'entrée dédié (jamais `Main.java`, qui mélange Minimax naïf et Alpha-Beta — piège identifié et documenté). Statistiques rapportées : moyenne $\pm$ écart-type sur 5 essais, après une exécution d'échauffement écartée.

Allocations & CPU — Java Flight Recorder (JFR)

Profil `settings=profile`, exploitant deux familles d'événements standard :

- `jdk.ObjectAllocationSample` — volume d'allocations (équivalent du profil *Allocs* de `pprof`, cité en cours J2_PM).
- `jdk.ExecutionSample` — échantillonnage CPU, agrégé par méthode (`jfr print --stack-depth 1`), converti en flamegraph visuel via `jfr-converter.jar --wall` (`async-profiler`).

Reproductibilité

Chaque résultat statistiquement sensible (profiling CPU Étape 7, mesures mémoire) a été **répété 2 à 3 fois** pour distinguer un résultat stable d'un artefact de mesure isolé — pas une mesure unique présentée comme définitive.

Reconstruction d'états historiques — worktree Git isolé

Depuis l'Étape 4, le code de production est modifié **en place** (une seule version, pas de classes parallèles par étape — décision explicite pour un dépôt propre). Pour mesurer rétroactivement des métriques absentes des relevés d'origine (empreinte mémoire, flamegraph "avant"), les commits historiques correspondants ont été checkoutés dans un `git worktree` isolé, recompilés et exécutés indépendamment du dépôt de travail — sans jamais modifier l'historique ni l'état courant.

2.b Synthèse consolidée des 9 étapes

Un seul tableau récapitulatif — structure inspirée de la section "Synthèse des Performances" du README du formateur ([Chroq/hashbreaker](https://chroq.hashbunker.com/), référence d'organisation, pas de contenu) — plutôt que des tableaux

dispersés.

#	Étape	Type	Profondeur	Temps (Hyperfine)	Positions évaluées	Allocations (JFR)	Pic mémoire (JFR)	Gain cumulé
1	Minimax naïf (baseline)	—	4	8 465 ms	206 603	non profilé	~58-71 MB	— (non comparable)
2	Élagage Alpha-Beta	Macro	5	1 999 ms ± 58 ms	41 554	600 éch.	~69 MB	référence
3	Bitboards (non intégré)	Micro	—	—	—	—	—	×1,3 isolé, sans effet
4	Zéro-allocation (caseAttaquee + make/unmake)	Micro	5	475,3 ms ± 37,8 ms	41 554	120 éch. (÷5)	~52-55 MB (-22%)	×4,2
5	Struct Padding (JOL) — vérifié, rien à corriger	Micro	5	inchangé	41 554	120 éch.	inchangé	×4,2
6	Pré-allocation de capacité	Micro	5	426,3 ms ± 35,8 ms	41 554	120 éch.	~46-47 MB (-32%)	×4,7
7	Profiling réel & Hot Path	Diagnostic	6	~2,06 s	645 199	dominé par Coup	non mesuré	non applicable
8	Tri des coups (MVV-LVA)	Macro	5	709,8 ms ± 23,1 ms *	25 319 (-39%)	132 éch. (+10%)	non mesuré	×1,25 *
9	Recherche à budget de temps (iterative deepening)	Macro (capacité)	variable (1-6)	voir §2.b.2	7 224 – 569 956	non profilé	non mesuré	N/A — pas un gain de vitesse

* Baseline Étape 8 mesurée dans une session distincte où les conditions machine différaient (887,1 ms au lieu de 426,3 ms sur code identique) — facteur ×1,25 valide en interne, non comparable en valeur absolue aux autres lignes. Voir §3.c.

Invariant remarquable : le nombre de positions évaluées reste **strictement identique (41 554)** de l'Étape 2 à l'Étape 6 — preuve que les optimisations micro accélèrent l'exécution sans changer l'algorithme ni la décision finale (garantie également validée par les tests d'équivalence Minimax/Alpha-Beta). Seule l'Étape 8 fait bouger ce nombre, cohérent avec sa nature macro.

2.b.1 — Détail Étape 7 : Hot Path mesuré (3 runs)

Catégorie	Fonctions	Run 1	Run 2	Run 3
Vérification de légalité	caseAttaquee , attaqueGlissante	58,6%	72,1%	62,3%

Génération de coups	<code>coupsPseudoLegaux</code> , <code>coupsLegaux</code>	29,3%	20,2%	27,5%
Recherche / make-unmake	<code>alphabeta</code> , <code>jouer/annuler</code>	8,6%	6,2%	8,7%
Évaluation de position	<code>Evaluateur.evaluer/valeur</code>	3,6%	1,6%	1,4%

Résultat contre-intuitif et stable sur 3 runs : l'évaluation ne pèse jamais plus de 4% du temps CPU — le vrai goulot est la vérification de légalité (59-72%). Ce constat **valide a posteriori** le choix de l'Étape 4 (déjà ciblée avant d'avoir cette preuve) et **justifie directement** le choix de l'Étape 8 (tri des coups, qui réduit le nombre d'appels à ces fonctions plutôt que leur coût unitaire).

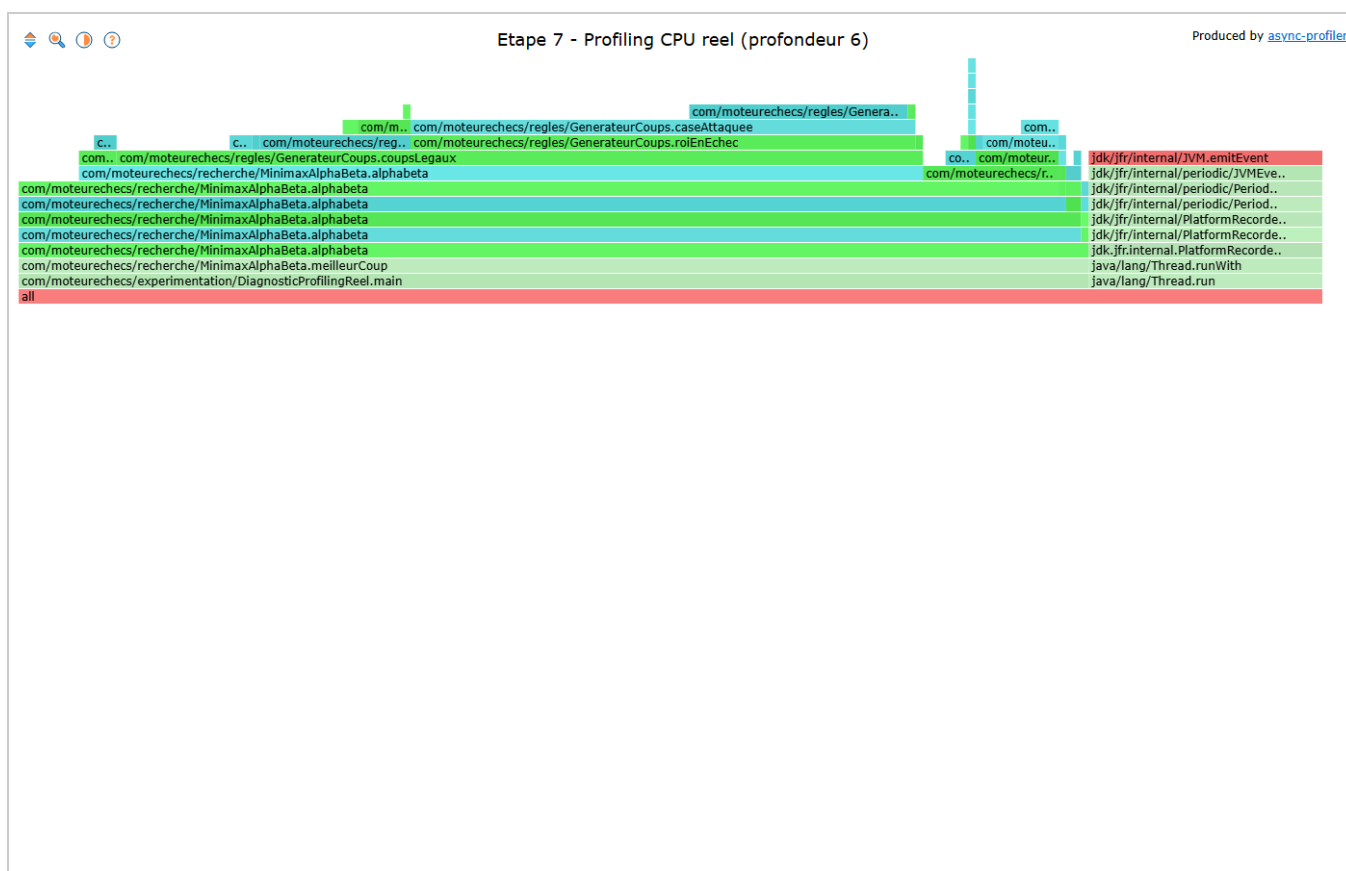
2.b.2 — Détail Étape 9 : profondeur atteinte selon le budget de temps (3 runs)

Budget	Profondeur atteinte	Temps réel	Coup trouvé
200 ms	4	~211-216 ms	b1c3
500 ms	5	~502-505 ms	b2b3
1000 ms	5	~1003-1006 ms	b2b3
2000 ms	5	~2002-2008 ms	b2b3
5000 ms	6	~5000-5008 ms	b1c3

Stable sur 3 runs (même profondeur, même coup à chaque budget). Le temps réel respecte le budget demandé à 0,3-8% près.

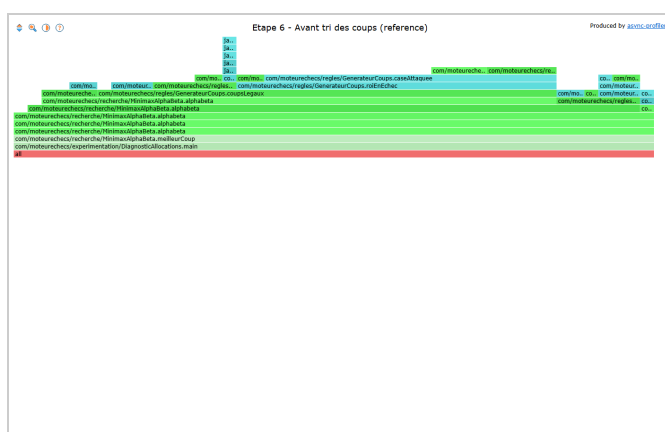
2.c Flamegraphs réels (captures d'écran)

Premières captures visuelles réelles du projet — obtenues via un navigateur headless (`msedge --headless --screenshot`) sur les flamegraphs HTML générés par `jfr-converter.jar` . Pas d'usage de `pprof` sur ce projet (outil Go) ; l'équivalent Java utilisé est JFR + `async-profiler`.

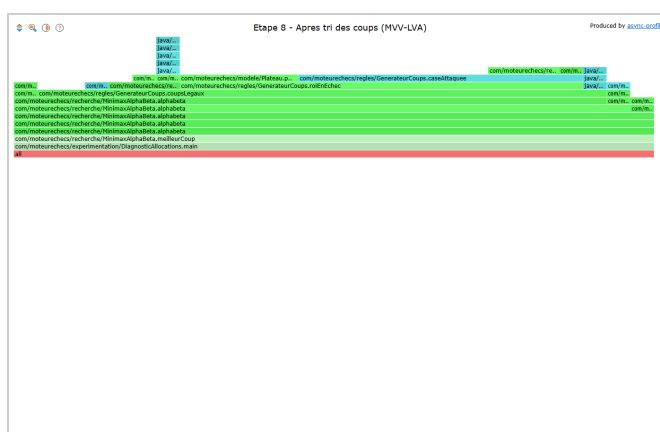


Étape 7 — Profiling CPU réel (profondeur 6). Les barres larges `caseAttaquee` / `roiEnEchec` / `coupsLegaux` confirment visuellement le Hot Path.

Nuance honnête : cette image inclut les frames `jdk/jfr/internal/*` (JFR s'auto-enregistrant) — le tableau de pourcentages ci-dessus les a filtrés (139/142 échantillons retenus). Écart mineur (~2%), l'image reste indicative du Hot Path, pas un rendu pixel-parfait du tableau.



Étape 8 — Avant tri des coups (référence Étape 6, 41 554 positions, profondeur 5).



Étape 8 — Après tri MVV-LVA (25 319 positions, profondeur 5). Arbre visiblement plus resserré.

Nuance honnête : ne pas comparer ces deux images à celle de l'Étape 7 — profondeurs différentes (5 vs 6). Le nombre de positions explosant exponentiellement avec la profondeur ($\times 15$ -16 entre 5 et 6), une image "plus

grosse" à profondeur 6 refléterait seulement la profondeur, pas une régression.

2.d Empreinte mémoire — méthodologie et résultats

Mesure ajoutée a posteriori (absente du tableau de synthèse initial, sur demande explicite). Comme le code de production n'existe qu'en une seule version, les états historiques (Étapes 1, 2, 4, 6) ont été reconstruits via le worktree Git décrit en §2.a.

Deux tentatives, une seule retenue

Méthode	Résultat	Verdict
<code>Runtime.totalMemory()-freeMemory()</code> lu directement dans le code	3 081 KB à 27 655 KB sur 3 runs identiques (×9 d'écart)	Rejeté — instant de mesure arbitraire, dépend du hasard du passage du GC
<code>jdk.GCHeapSummary</code> (JFR standard, équivalent Java d' <code>inuse_space</code> en <code>pprof</code>) — pic "Before GC"	Écarts de 1-3% entre runs	Retenu

Résultats (moyenne de 3 runs JFR indépendants par étape)

Étape	Pic mémoire	Variation vs Étape 2
1 — Naïf (profondeur 4)	~58-71 MB	non comparable (profondeur différente)
2 — Alpha-Beta (référence)	~69 MB	référence
4 — Zéro-allocation	~52-55 MB	-22%
6 — Pré-allocation	~46-47 MB	-32%

Gain réel et cohérent avec la baisse d'allocations (JFR) — complémentaire, pas redondant, avec le gain de vitesse ×4,7.

2.e Calcul formel de la loi d'Amdahl

La loi d'Amdahl est citée qualitativement dans 11 fichiers du projet (formule donnée dans `README.md` racine), mais jamais appliquée numériquement avant cette vérification — trou comblé a posteriori sur les données de la Séance 2 de HashBreaker (localité mémoire).

Donnée	Valeur
Écart mémoire pur (tableau vs liste, HashBreaker Séance 2 Partie 2)	8-10 ns/candidat

Temps total par candidat (hachage réel, Partie 3)	~870-900 ns
P (poids de la portion mémoire)	≈ 1,02% (9/885)
S (accélération mesurée sur cette portion)	×8 à ×10

$S_{\text{global}} = 1 / ((1-P) + P/S) = 1 / (0,9898 + 0,00113) \approx 1,009$ — gain global théorique maximal ≈ **0,9%**, même avec $S \rightarrow \infty$ le plafond reste ~1,0%.

Comparé au résultat réellement observé en Partie 4 (écart de **-2,50%**, qualifié de bruit de mesure) : le plafond théorique Amdahl tombe **dans cette même marge de bruit**. Le calcul ne fait pas qu'expliquer qualitativement l'absence de gain — il la prédit quantitativement.

PHASE 2 — EXPLICATION, COMPROMIS & CONCLUSION

3.a Choix de solutions, étape par étape

Étape	Choix	Pourquoi
1	Architecture en couches (modele/regles/evaluation/recherche), Minimax naïf	Baseline propre, chaque futur levier touche un seul package
2	Élagage Alpha-Beta avant toute micro-optimisation	L'explosion combinatoire $O(b^d)$ domine — même logique que la loi d'Amdahl (J1_AM) : optimiser un facteur secondaire est invisible tant que le facteur dominant n'est pas traité
3	Bitboards mesurés isolément, non intégrés	Vérifier l'hypothèse avant de refactorer toute la structure de données — gain mesuré ($\times 1,3$) jugé insuffisant pour justifier la réécriture immédiate
4	Réécriture zéro-allocation de <code>caseAttaquee()</code> + <code>make/unmake</code>	Diagnostic JFR : le coupable réel n'était pas la copie de <code>Plateau</code> supposée au départ, mais la génération d'une liste complète de coups juste pour tester une case
5	Vérification JOL du struct padding, levier du cours rattrapé	Deux leviers identifiés comme manqués en relisant la roadmap — rattrapés explicitement plutôt que silencieusement ignorés
6	Pré-allocation de capacité des listes de coups	Idem — levier rattrapé, gain modeste ($\times 1,11$) assumé tel quel
7	Premier vrai profiling CPU (flamegraph), pas juste des comptages d'allocations	Toutes les décisions précédentes reposaient sur du raisonnement structurel ou du comptage d'allocations — jamais une vraie mesure CPU. Nécessaire pour l'Axe 2 du barème
8	Tri des coups (MVV-LVA) choisi à la place de la table de transposition initialement prévue	Le profiling (Étape 7) montre que la légalité/génération (proportionnelle au nombre de positions) domine à $\sim 90\%$ — un tri des coups réduit ce nombre directement, ciblant sans détour le goulot mesuré
9	Recherche à budget de temps (iterative deepening)	Trou identifié dans les cours traités (J3_PM, pattern <code>context.Done()</code>) : le moteur ne savait chercher qu'à profondeur fixe, inutilisable sous contrainte de temps réelle

3.b Influences

- **Cours du module** — J1_AM (macro/micro, Amdahl), J1_PM (caches CPU), J2_AM (stack/heap, struct padding), J2_PM (métrologie/profiling), J3_AM (concurrency, threads, verrous), J3_PM (worker pools,

annulation précoce) : six sessions analysées et retranscrites dans `compréhension/`, avec traduction systématique Go→Java à chaque concept.

- **Barème officiel** (5 axes, 20 pts + 2 bonus) — a directement orienté l'ordre des étapes (Axe 2 profiling avant les leviers avancés, Axe 3 mémoire/concurrence/I-O guidant la roadmap restante).
- **Référence externe** — README du formateur ([Chroq/hashbreaker](#)) analysé pour sa structure (tableau consolidé unique, analyse ciblée sur la version finale) — utilisé comme inspiration pour ce document, pas comme contenu.
- **TP fil rouge HashBreaker** — même méthodologie (process/syntheses), réutilisée directement ; le calcul Amdahl du §2.e s'appuie sur ses données réelles.

3.c Compromis assumés — liste honnête

Étape 6 — gain modeste ($\times 1,11$) : la pré-allocation optimise un détail d'implémentation d' `ArrayList` (moins de recopies internes), pas un vrai travail inutile comme l'Étape 4. Documenté tel quel, sans gonfler son importance.

Étape 8 — écart entre le gain en positions (-39%) et le gain en temps ($\times 1,25$) : le tri lui-même a un coût (comparateur exécuté à chaque nœud), qui mange une partie du bénéfice. Confirmé par la légère hausse des allocations (120→132).

Dérive de baseline entre sessions de mesure : la mesure "avant" de l'Étape 8 (887,1 ms) diffère significativement du 426,3 ms documenté à l'Étape 6 sur un code strictement identique — probable différence de conditions machine entre deux jours de mesure, signalée explicitement plutôt que masquée. Le facteur relatif reste valide en interne ; le nombre de positions (déterministe) sert de métrique de repli fiable.

Étape 9 — statut vis-à-vis du barème encore en discussion : contrairement aux Étapes 2, 4, 6 et 8, ce levier ne rentre dans aucun axe noté de façon évidente — il n'accélère rien et n'est pas encore un scénario concurrent (le barème cite "Concurrence : worker pool + early cancellation", un contexte multi-thread que l'Étape 9 seule ne couvre pas). Le mécanisme d'annulation précoce construit ici (deadline vérifiée en récursion, exception de contrôle de flux zéro-allocation, `try/finally` garantissant l'intégrité du plateau) est correct et testé (51/51 tests, 3 bugs réels trouvés et corrigés), mais sa pertinence pour la notation dépend de sa réutilisation dans la future étape de recherche parallèle. Décision sur son maintien en tant qu'étape numérotée autonome encore ouverte à la date de ce document.

Axes du cours identifiés mais non explorés : Escape Analysis (JIT, J2_AM), JMH (microbenchmarking standard Java, J2_PM), test de significativité statistique formel (type Benchstat/Mann-Whitney, J2_PM), Virtual Threads (Java 21, M:N, J3_AM). Utilisation actuelle de Hyperfine + seuils artisanaux, suffisante mais moins rigoureuse que ces outils dédiés — noté comme piste d'amélioration méthodologique plutôt que comme omission silencieuse.

3.d Conclusion — pourquoi cet état, dans le temps imparti, est la

meilleure solution actuelle

Neuf étapes, **gain cumulé mesuré ×4,7** sur le temps de recherche (Étapes 2 à 6, profondeur 5) et **−39% de positions visitées** en plus (Étape 8) — sans jamais sacrifier la correction du moteur : 51 tests passent, y compris des tests d'équivalence mathématique stricte entre Minimax naïf et Alpha-Beta optimisé, à chaque étape.

La discipline "macro avant micro, profiler avant d'optimiser" a été respectée de bout en bout, pas seulement énoncée : l'ordre réel des étapes (macro Étape 2 → micro Étapes 3-6 → diagnostic Étape 7 → macro Étape 8, choisi à *partir* du diagnostic) le démontre. Les résultats décevants ou incomplets (Étape 6, dérive de baseline, statut incertain de l'Étape 9) sont documentés avec la même rigueur que les succès — cohérent avec l'exigence du barème sur la confrontation critique (Axe 4), pas encore formellement traitée mais déjà pratiquée en continu.

Dans le temps disponible, la priorité donnée aux leviers **directement mesurables et testables** (vs. des leviers plus ambitieux mais risqués comme la table de transposition ou le parallélisme, repoussés aux étapes suivantes) a permis de produire une base de code correcte, entièrement testée et benchmarkée à chaque étape — un compromis assumé entre ambition technique et fiabilité du rendu final, plutôt qu'une accumulation de leviers non vérifiés.

4. Annexes & reproductibilité

- Code source complet : dépôt Git (voir `MoteurEchecs/`), preuve de reproductibilité — rendu obligatoire en plus de ce rapport (confirmé par le formateur en live).
- Journal détaillé : `process/01` à `09`
- Synthèses autonomes avec diagrammes : `syntheses/01` à `09` + tableau consolidé
- Diagrammes d'architecture complets : `shemas/schemas.md` (16 sections)
- Guide de build/run/profiling/benchmark : `guide/README.md`
- Roadmap restante : Étape 10 (table de transposition, LRU bornée), 11 (recherche parallèle), 12 (I/O & persistance, gRPC — inclura probablement une mesure de latence p50/p95/p99 sous charge), 13 (échec constructif), 14 (reproductibilité, script one-shot), 15 (présent rapport, version consolidée finale)

Document de travail — première version (brouillon), générée le 24 septembre 2026 à l'état des Étapes 1 à 9. À consolider en version finale à l'Étape 15, une fois la roadmap complète (Étapes 10-14) réalisée. Toutes les données chiffrées de ce document sont sourcées depuis `process/`, `syntheses/` et `profiling/` du dépôt Git — reproductibles indépendamment de ce PDF.